

Lab 05

Mariana Zamth Feltrin

```
#pacotes necessários  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.2.1      v readr      2.2.0  
v forcats    1.0.1      v stringr    1.6.0  
v ggplot2    4.0.3      v tibble     3.3.1  
v lubridate  1.9.5      v tidyr      1.3.2  
v purrr      1.2.2  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(readr)  
library(leaflet)  
library(geosphere)
```

```
analisa_aeronave <- function(tail_number, arquivo) {  
  
  # 1. LEITURA DOS AEROPORTOS  
  arquivo_airports <- read_csv("airports.csv", show_col_types = FALSE) %>%  
    select(IATA_CODE, LATITUDE, LONGITUDE)  
  
  # 2. LEITURA DO FLIGHTS POR PARTES  
  tabela_voos <- read_csv_chunked(  
    arquivo,  
    callback = DataFrameCallback$new(function(chunk, pos) {  
      chunk %>% filter(TAIL_NUMBER == tail_number)  
    }),  
    chunk_size = 50000,  
  )  
}
```

```

    show_col_types = FALSE
  )

  if (nrow(tabela_voos) == 0) {
    stop("Nenhum registro encontrado para o TAIL_NUMBER informado.")
  }

  # Detecta os nomes reais das colunas de origem/destino no flights.csv
  col_origem <- if ("ORIGIN_AIRPORT" %in% names(tabela_voos)) "ORIGIN_AIRPORT" else "ORIGIN"
  col_destino <- if ("DESTINATION_AIRPORT" %in% names(tabela_voos)) "DESTINATION_AIRPORT" else "DESTINATION"

  # 3. CONVERSÃO E CRIAÇÃO DA DATA/HORA
  tabela_voos <- tabela_voos %>%
    mutate(
      DEPARTURE_TIME_NUM = as.numeric(DEPARTURE_TIME),
      data_voo = make_date(YEAR, MONTH, DAY),
      hora_partida_minutos = if_else(
        is.na(DEPARTURE_TIME_NUM),
        NA_real_,
        (DEPARTURE_TIME_NUM %% 100) * 60 + (DEPARTURE_TIME_NUM %% 100)
      )
    ) %>%
    arrange(data_voo, hora_partida_minutos)

  # 4. JUNÇÃO OTIMIZADA COM AS COORDENADAS
  tabela_voos <- tabela_voos %>%
    left_join(
      arquivo_airports,
      by = c("ORIGIN_AIRPORT" = "IATA_CODE")
    ) %>%
    rename(
      latitude_origem = LATITUDE,
      longitude_origem = LONGITUDE
    ) %>%
    left_join(
      arquivo_airports,
      by = c("DESTINATION_AIRPORT" = "IATA_CODE")
    ) %>%
    rename(
      latitude_destino = LATITUDE,
      longitude_destino = LONGITUDE
    )

```

```

# 5. CÁLCULO DE DISTÂNCIA E VELOCIDADE MÉDIA
tabela_voos <- tabela_voos %>%
  mutate(
    AIR_TIME = as.numeric(AIR_TIME),
    distancia_km = if_else(
      !is.na(longitude_origem) & !is.na(latitude_origem) &
      !is.na(longitude_destino) & !is.na(latitude_destino),
      distHaversine(
        cbind(longitude_origem, latitude_origem),
        cbind(longitude_destino, latitude_destino)
      ) / 1000,
      NA_real_
    ),
    velocidade_media_kmh = if_else(
      !is.na(AIR_TIME) & AIR_TIME > 0,
      distancia_km / (AIR_TIME / 60),
      NA_real_
    )
  )

# 6. CRIAÇÃO DO MAPA INTERATIVO
longs <- c(tabela_voos$longitude_origem, tabela_voos$longitude_destino)
lats <- c(tabela_voos$latitude_origem, tabela_voos$latitude_destino)

mapa <- leaflet() %>%
  addTiles() %>%
  setView(
    lng = mean(longs, na.rm = TRUE),
    lat = mean(lats, na.rm = TRUE),
    zoom = 4
  )

for (i in seq_len(nrow(tabela_voos))) {
  voo <- tabela_voos[i, ]

  if (!is.na(voo$longitude_origem) && !is.na(voo$latitude_destino)) {
    espessura <- if_else(
      is.na(voo$velocidade_media_kmh), 2,
      pmax(2, pmin(8, voo$velocidade_media_kmh / 100))
    )

    texto_popup <- paste0(

```

```

    "<b>Data:</b> ", voo$data_voo,
    "<br><b>Trecho:</b> ", voo[[col_origem]], " ", voo[[col_destino]],
    "<br><b>Distância:</b> ", round(voo$distancia_km, 1), " km",
    "<br><b>Velocidade Média:</b> ", round(voo$velocidade_media_kmh, 1), " km/h"
  )

  mapa <- mapa %>%
    addPolylines(
      lng = c(voo$longitude_origem, voo$longitude_destino),
      lat = c(voo$latitude_origem, voo$latitude_destino),
      color = "#1E88E5",
      weight = espessura,
      opacity = 0.6,
      popup = texto_popup
    )
  }
}

# Marcadores dos aeroportos
aeroportos_visitados <- bind_rows(
  tabela_voos %>% select(codigo = all_of(col_origem), lat = latitude_origem, lng = longitude_origem),
  tabela_voos %>% select(codigo = all_of(col_destino), lat = latitude_destino, lng = longitude_destino)
) %>%
  distinct(codigo, .keep_all = TRUE) %>%
  filter(!is.na(lat), !is.na(lng))

mapa <- mapa %>%
  addCircleMarkers(
    data = aeroportos_visitados,
    lng = ~lng,
    lat = ~lat,
    radius = 5,
    color = "#D81B60",
    fillColor = "#FFC107",
    fillOpacity = 0.9,
    popup = ~paste0("<b>Aeroporto:</b> ", codigo)
  )

return(list(
  tabela = tabela_voos,
  grafico = mapa
))

```

```
}
```

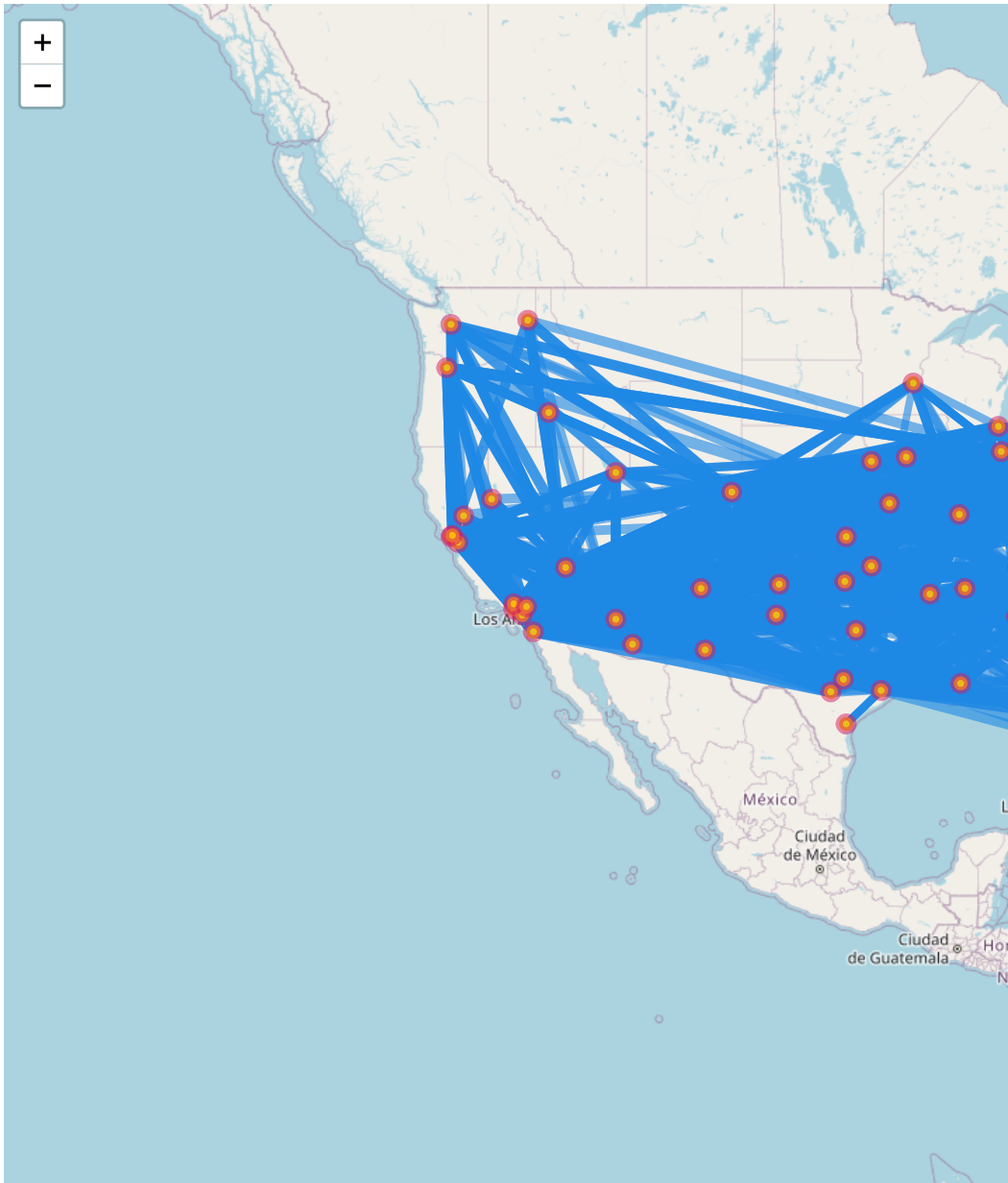
```
# 1. Carregue as bibliotecas necessárias
library(htmlwidgets)
library(webshot2)

# 2. Gere o resultado através da sua função
resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("flights.csv.zip")
)

# 3. Salva o HTML temporário e exporta para PDF
saveWidget(resultado$grafico, file = "mapa_temp.html")

webshot(
  url = "mapa_temp.html",
  file = "mapa_aeronave.pdf",
  vwidth = 1200,
  vheight = 800
)
```

file:///\\smb/ra193739/Downloads/ME315/Aula 05/mapa_temp.html screenshot completed



Leaflet | © OpenStreetMap, ODbL

```
print(resultado$tabela)
```

```
# A tibble: 1,938 x 40
```

	YEAR	MONTH	DAY	DAY_OF_WEEK	AIRLINE	FLIGHT_NUMBER	TAIL_NUMBER
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>	<dbl>	<chr>
1	2015	1	1	4	WN	1762	N431WN
2	2015	1	1	4	WN	3564	N431WN
3	2015	1	1	4	WN	3564	N431WN
4	2015	1	1	4	WN	3564	N431WN
5	2015	1	2	5	WN	1793	N431WN
6	2015	1	2	5	WN	1793	N431WN
7	2015	1	2	5	WN	1793	N431WN
8	2015	1	2	5	WN	1793	N431WN
9	2015	1	2	5	WN	1793	N431WN
10	2015	1	2	5	WN	3144	N431WN

```
# i 1,928 more rows
```

```
# i 33 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,  
# SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,  
# TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,  
# AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,  
# SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,  
# DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...
```

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-14 09:36:53 -03"
```